

Exposing a Local ASP.NET Core App with Cloudflare Tunnel

Project: ElyraBd | Setup Documentation

1. Overview

This document describes how the ElyraBd ASP.NET Core Web application, running locally in Visual Studio, was configured to run correctly and then exposed to the public internet using Cloudflare's free Quick Tunnel (trycloudflare.com), with no port forwarding or paid domain required.

2. Fixing the Startup Project Error

When first running the solution, Visual Studio displayed the error: "A project with an Output Type of Class Library cannot be started directly." This occurred because the startup project was set to a Class Library (ElyraBd.Application) instead of the executable Web project (ElyraBd.Web).

Resolution Steps

- Confirmed all project files existed using: `dir /s /b *.csproj`

```
dir /s /b *.csproj
```

- Verified ElyraBd.Web.csproj was already registered in the solution file (ElyraBd.slnx) using:

```
dotnet sln ElyraBd.slnx add ElyraBd.Web\ElyraBd.Web.csproj
```

- Closed and reopened the solution in Visual Studio to force it to reload all referenced projects.
- In the Startup Project dropdown (toolbar) or via Solution Explorer, selected ElyraBd.Web and set it as the Startup Project.
- Ran the project successfully with F5, which launched the app at `https://localhost:7025`.

3. Installing Cloudflare Tunnel (cloudflared)

Cloudflared was installed on Windows using the Windows Package Manager:

```
winget install --id Cloudflare.cloudflared
```

Alternatively, the executable can be downloaded directly from the official GitHub releases page: <https://github.com/cloudflare/cloudflared/releases>

4. Creating the Quick Tunnel

With the ASP.NET application running locally on `https://localhost:7025`, the following command was run in a terminal:

```
cloudflared tunnel --url https://localhost:7025
```

If the local site uses a self-signed HTTPS development certificate, the following flag can be added to bypass certificate verification for the tunnel connection:

```
cloudflared tunnel --url https://localhost:7025 --no-tls-verify
```

Cloudflared performed connectivity pre-checks (DNS resolution, UDP/QUIC, TCP, and Cloudflare API reachability), all of which passed, and generated a public URL:

```
https://foo-explosion-fallen-towers.trycloudflare.com
```

5. Remote Access

Once the tunnel was active, the application became accessible from any device with internet access, simply by visiting the generated public URL in a browser. No router configuration, port forwarding, or firewall changes were required.

Requirements to Keep the Tunnel Active

- The ASP.NET Core application must remain running locally (`https://localhost:7025`).
- The terminal window running the cloudflared tunnel command must remain open.
- Closing either process will immediately break remote access.

6. Limitations of the Free Quick Tunnel

- No uptime guarantee — intended for testing, demos, and experimentation, not production use.
- The public URL is randomly generated and changes every time the tunnel is restarted.
- For a permanent, fixed subdomain, a free Cloudflare account with a registered domain and a named tunnel configuration is required instead of the quick `--url` method.

7. Summary

The ElyraBd Web application was successfully corrected to launch as the proper startup project in Visual Studio and was exposed to the public internet in minutes using Cloudflare's free Quick Tunnel service, enabling remote access without any paid infrastructure or complex network configuration.